

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

LAB 5- github link

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

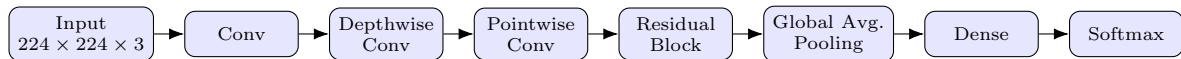
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	3.0869	17.66	5	112.2
MOMENTUM	0.6168	82.07	5	114.0
RMSPROP	0.0371	88.99	4	110.1
ADAM	0.0428	88.72	5	108.9

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\bar{A} \pm SD.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	Mean $\pm$ SD
C1	88.18	85.33	88.50	87.34 $\pm$ 1.43
C2	88.26	87.20	88.50	87.99 $\pm$ 0.56
C3	84.52	86.72	87.44	86.22 $\pm$ 1.24
C4	21.68	21.92	20.31	21.30 $\pm$ 0.71

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy (%)	8.799000e+01
CV Standard Deviation	5.600000e-01
Test Accuracy (%)	8.648000e+01
Precision	8.685000e-01
Recall	8.648000e-01
F1-score	8.632000e-01
Training Time (s)	5.170000e+02
Number of Parameters	2.431845e+06

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline (No Reg., Adam, He init, frozen base)	—	—	89.54%	—
Best Initialization (Random)	—	—	89.27%	—
Best Regularization (Batch Normalization)	—	—	90.35%	—
Best Optimizer (RMSPROP)	—	—	88.99%	—
Best Hyperparameters (Sec. 9 sweep)	—	—	89.95%	—
Fine-Tuned Model / Final Selected (C2)	87.99%	0.56	86.48%	517.0s

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters? parameters are learnable values which can learnt from each epoch or gradient meanwhile hyperparameters are fixed values that are set even before training
2. Why is weight initialization important? weight initialization plays an important factor whether the model will converge quickly or not. poor initalization can cause vanishing or exploding gradients, in some casesit will learn symmetrically and thus no distinction in parameter values
3. Why can zero initialization be problematic for neural networks? if the initialization is all starting from 0 then every layer computes the same value, hence even in backpropogation all the error will be distributed equally which will lead to learning symmetric weights
4. Compare Xavier and He initialization. xavier initialization is meant for sigmoid and tanh activation functions and start with a lower range of values for parameter initialization meanwhile he intitialization is meant for relu activation functions and initialize to a relatively larger values for the weights
5. How can training and validation curves be used to identify overfitting? from the two graphs, if we see the training curve constant increasing and the validation having plateau or decline, we can assume there is overfitting. if the gap between these 2 cirves is large then we can say that the model is overfitting and is worse but if the gap is less then the model is said to be performing well
6. How does Dropout reduce overfitting? dropout randomly drops a few neuron to force the model to not completely rely on a single path for the training and learning, this process running thorough multiple epochs makes it to be a subnet of models, increasing generalization and robustness
7. What is the purpose of Batch Normalization? batch normaliztion is a process that happens during training to ensure that the activation values across all layers are in similar range ensuring stability to the model, otherwise it may lead to divergence due to vanishing or exploding gradients
8. Explain the numerical Batch Normalization example. For $x = [2, 4, 6, 8]$, the mean is $\mu_B = 5$ and the variance is $\sigma_B^2 = 5$. Therefore,

$$\sqrt{\sigma_B^2 + \epsilon} \approx 2.236, \quad \hat{x}_i = \frac{x_i - \mu_B}{2.236}.$$

The normalized values are

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342],$$

which are centered at zero with unit variance. With $\gamma = 1$ and $\beta = 0$, the output is exactly $\hat{x}$.

9. What are the roles of γ and β ? these values are learnale parameters that adjust the normalization per channel to help represent useful features
10. Compare SGD, Momentum, RMSProp and Adam. one, SGD takes fixed steps and updates weights accordingly. second, momentum is like a moving average that is used to accelated along the consistent direction. third, rmsprop is used to ensure even if there are large gradients, the steps taken are not too big which may move over the optima, hence if the gradient is large, the step is less and vice versa. lastly adam is the combination of momentum and rmsprop, giving adaptive per parameter learning rates, which makes it robust and best out of these 4
11. What happens when the learning rate is too large? instead of converging smoothly, it may overshoot over the minima, can oscillate and even have dovergence in some cases due to unstability.
12. What happens when the learning rate is too small? may need to go through huge number of epoch to find minima

as it might take really long time for convergence, in some cases it can even get stuck at local minima.

13. What is the effect of increasing batch size? increasing the batch size may lead to losing the stochastic noise that helps escape minima and thus can lead to slightly less generalization and may need more epochs to converge
14. Explain stride and padding. stride is the step size moved across in the kernel, padding is the extra layers around the input's border, which is mostly used to preserve the spatial output size
15. Why is MobileNetV2 computationally efficient? MobileNet is lightweight because it uses depthwise separable convolutions and inverted residual blocks, which greatly reduce the number of parameters, computation, and memory needed for mobile/CPU devices.
16. What is depthwise separable convolution? Depthwise separable convolution is a faster and lighter way of doing convolution in which depthwise convolution finds spatial features separately in each channel, and then a 1 cross 1 matrix pointwise convolution combines those channels, reducing computation and parameters
17. What is transfer learning? transfer learning is where a pretrained model is used without making it from scratch with random weight initialization. this is better as it would have already learned like general visual features like borders, textures etc. , making it easier as it only needs to learn less parameters than actually needed
18. Differentiate feature extraction and fine-tuning. feature extraction doesn't unfreeze the layers, it just trains the new classifier head that is usually plugged in last to the output features layer, fine tuning is where you unfreeze the last few layers and then use a small learning rate to train them so they can be specifically learnt to the targeted dataset or task
19. Why is a smaller learning rate generally used during fine-tuning? large learning rate causes huge gradients that may destroy the prior knowledge the pretrained model may have had and thus may lead to decline in the performance of the model as the pretrained model is mostly frozen with useful weights and features learnt from the previous training/dataset/
20. Why is K-Fold Cross-Validation useful for hyperparameter selection? a single validation may give biased results which may be misleading, thus k folds ensures it takes the values k different times and then averages it out, which would give you a more reliable answer than a single validation
21. Why must the test set remain untouched during tuning? the test set should remain untouched during training as it is used to evaluate the performance of the trained model on an unseen data to give an unbiased performance as it can memorize the dataset during the training which leads to poor generalization, hence unseen test data is used to for fair and unbiased evaluation on the performance
22. Why should mean and standard deviation both be reported? The mean tells us the model's average performance, while the standard deviation tells us how consistent that performance is across different folds, so a low SD means the model is more reliable and stable.
23. Is the highest validation accuracy always sufficient to select a model? No. A single highest validation-accuracy number can result from a lucky data split, can hide high variance which can come from a configuration with much higher computational cost for a marginal gain, or can even reflect a validation set that leaked into hyperparameter tuning too many times

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Configuration	Mean CV Accuracy (%)	SD
E1 (aggressive fine-tune, low dropout)	41.93	2.06
E2 (feature-extraction, high dropout, RMSProp)	85.25	1.28
Previously selected: C2	87.99	0.56

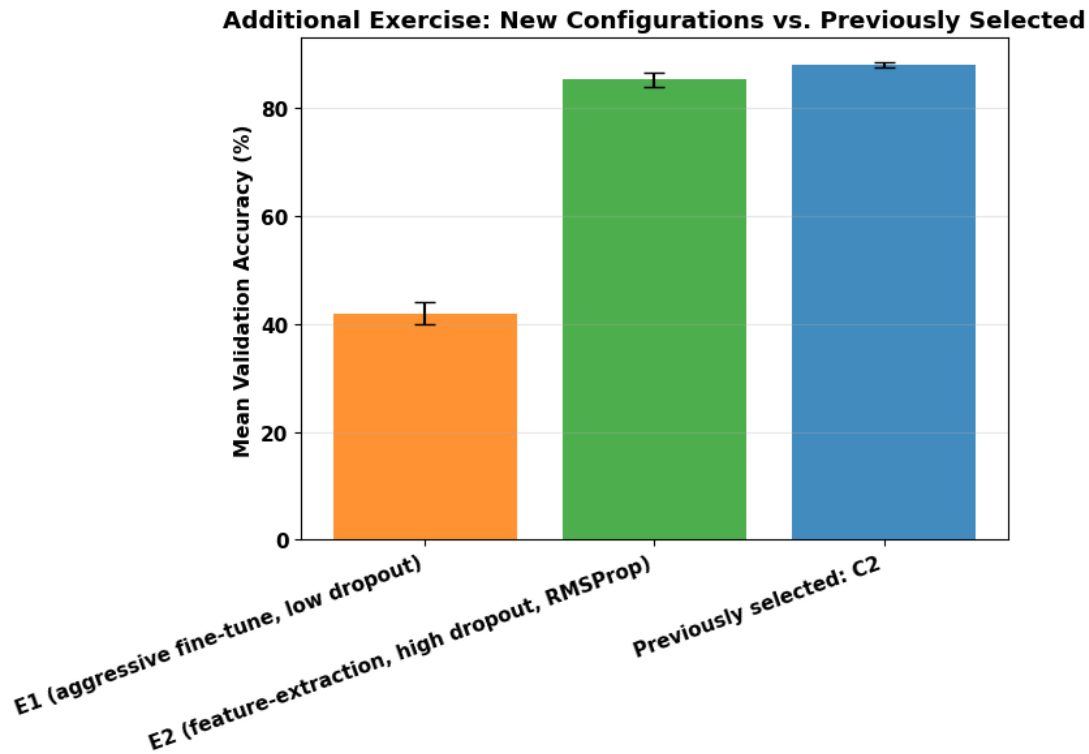


Figure 1: Cross-validation comparison of the additional configurations and C2.

C2 retains the highest mean validation accuracy at 87.99% and the lowest variability with an SD of 0.56. E2 performs reasonably well but is less accurate and consistent, while the aggressive fine-tuning used by E1 causes a large performance drop. Therefore, C2 remains the preferable and most reliable configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>

19. plots

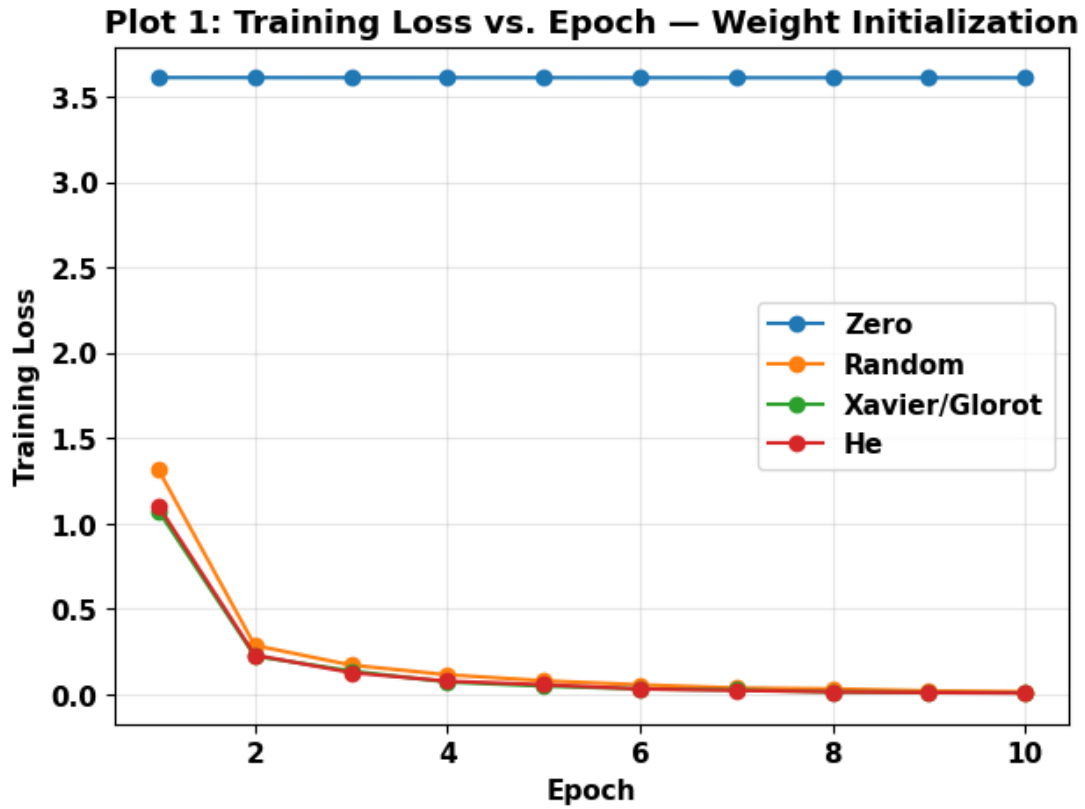


Figure 2: Training loss versus epoch for the weight initialization methods.

Inference: The plot compares training loss across ten epochs for zero, random, Xavier/Glorot, and He initialization. Random, Xavier/Glorot, and He initialization rapidly reduce the loss to nearly zero, whereas zero initialization remains at a high, constant loss. This occurs because identical zero weights preserve symmetry and prevent neurons from learning distinct features, while nonzero initializations allow effective gradient updates.

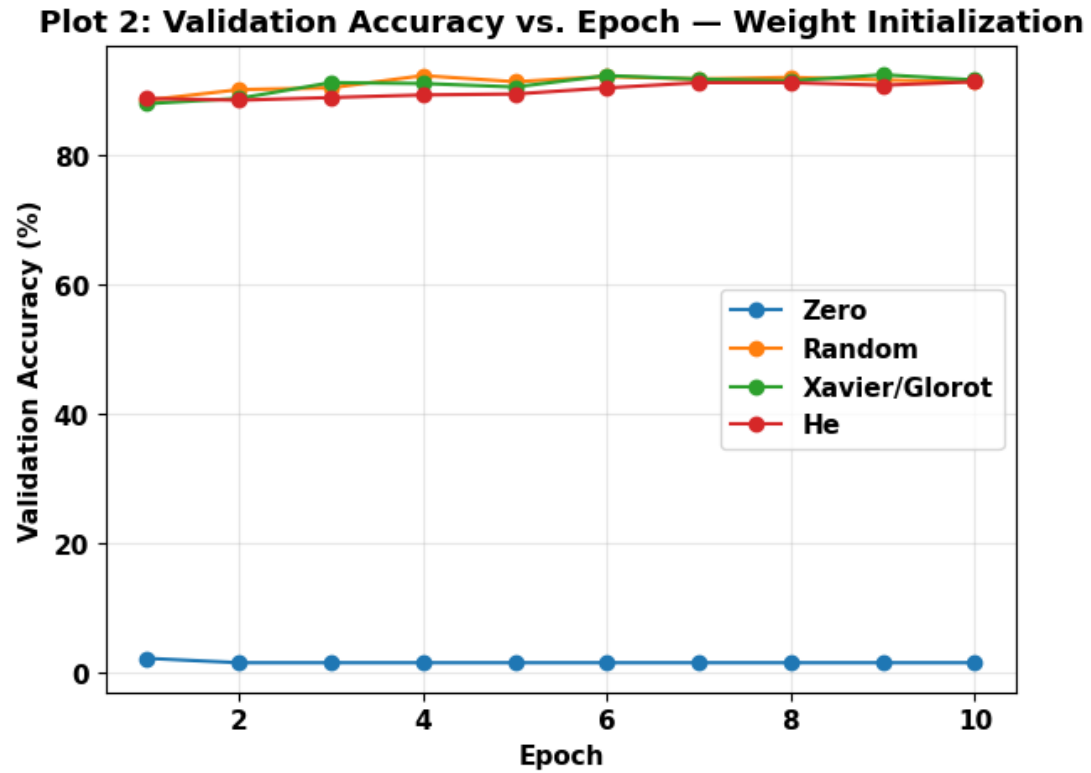


Figure 3: Validation accuracy versus epoch for the weight initialization methods.

Inference: The plot compares validation accuracy across ten epochs for the four initialization methods. Random, Xavier/Glorot, and He initialization remain high at approximately 88–93%, while zero initialization stays near 2%. The nonzero methods break symmetry and support useful feature learning; their small fluctuations reflect normal variation during training.

Plot 3: Training and Validation Accuracy vs. Epoch — Regularization

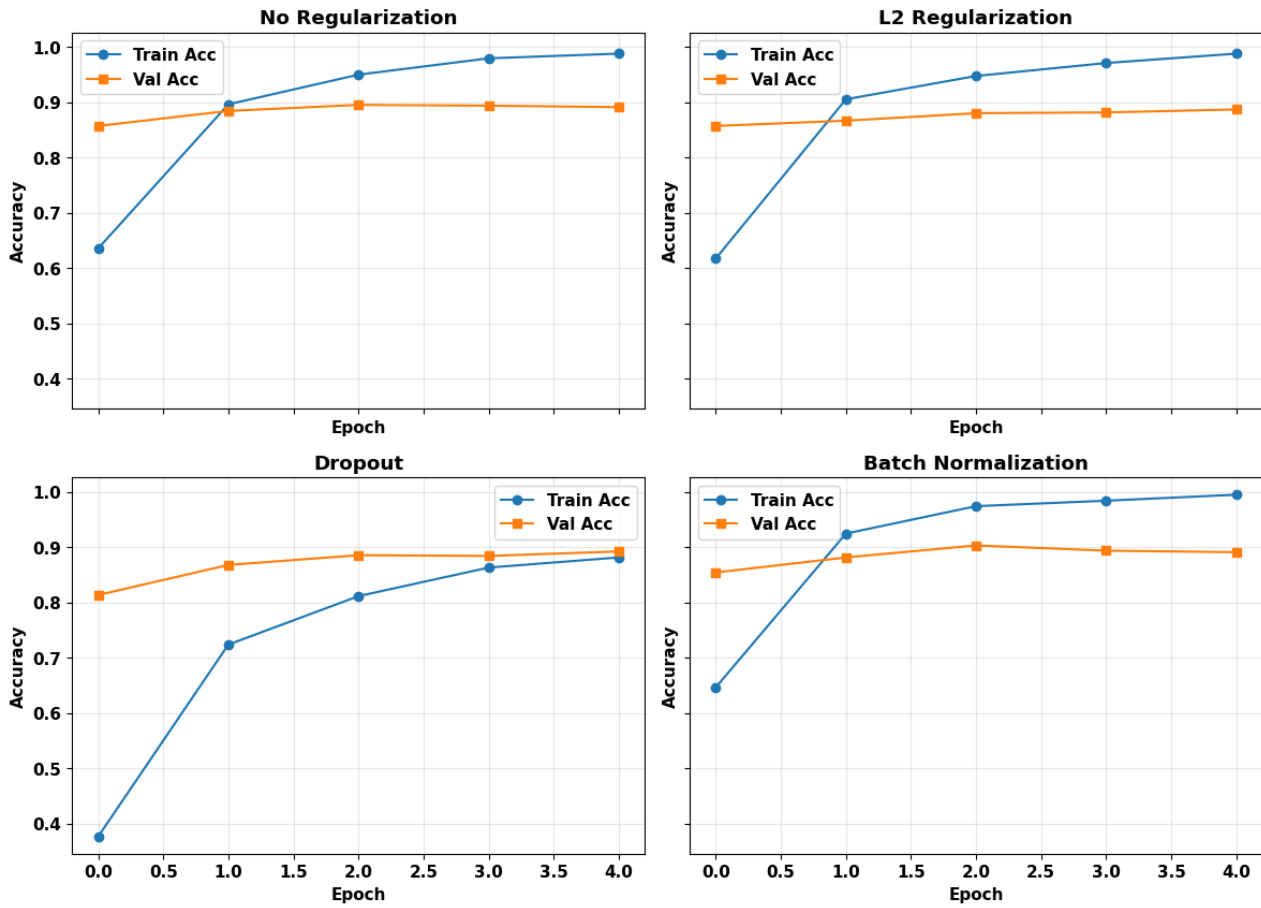


Figure 4: Training and validation accuracy for different regularization methods.

Inference: The plot compares training and validation accuracy for no regularization, L2 regularization, Dropout, and Batch Normalization. The first, second, and fourth settings develop a visible generalization gap, while Dropout keeps the curves closest by the final epoch. Dropout limits neuron co-adaptation and therefore reduces the tendency to overfit the training data.

Plot 4: Training and Validation Loss vs. Epoch — Regularization

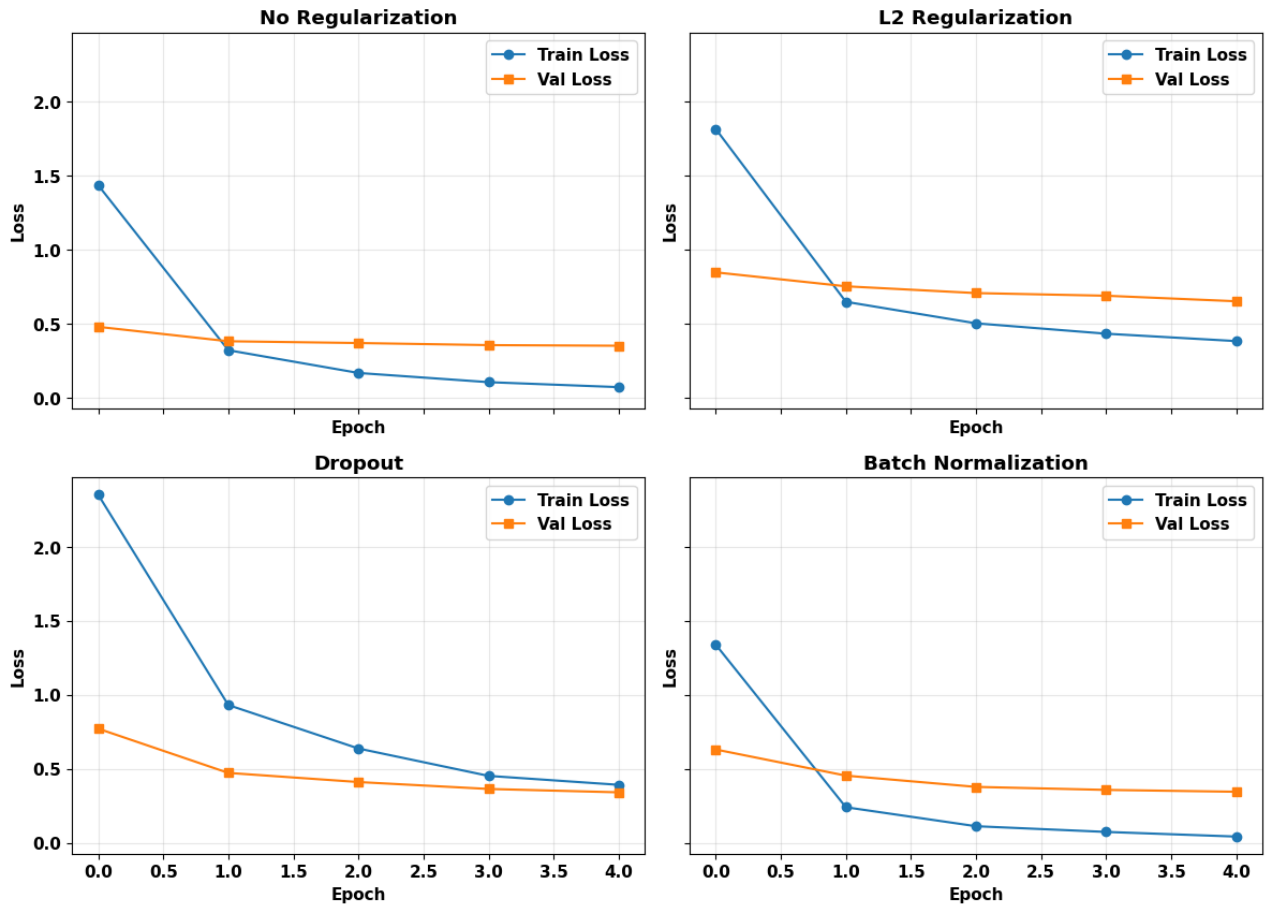


Figure 5: Training and validation loss for different regularization methods.

Inference: Training loss decreases for every method, but validation loss falls more slowly and remains higher. No regularization and Batch Normalization show the largest final loss gaps, whereas Dropout gives more balanced curves. Regularization discourages excessive fitting to the training samples and can therefore improve generalization.

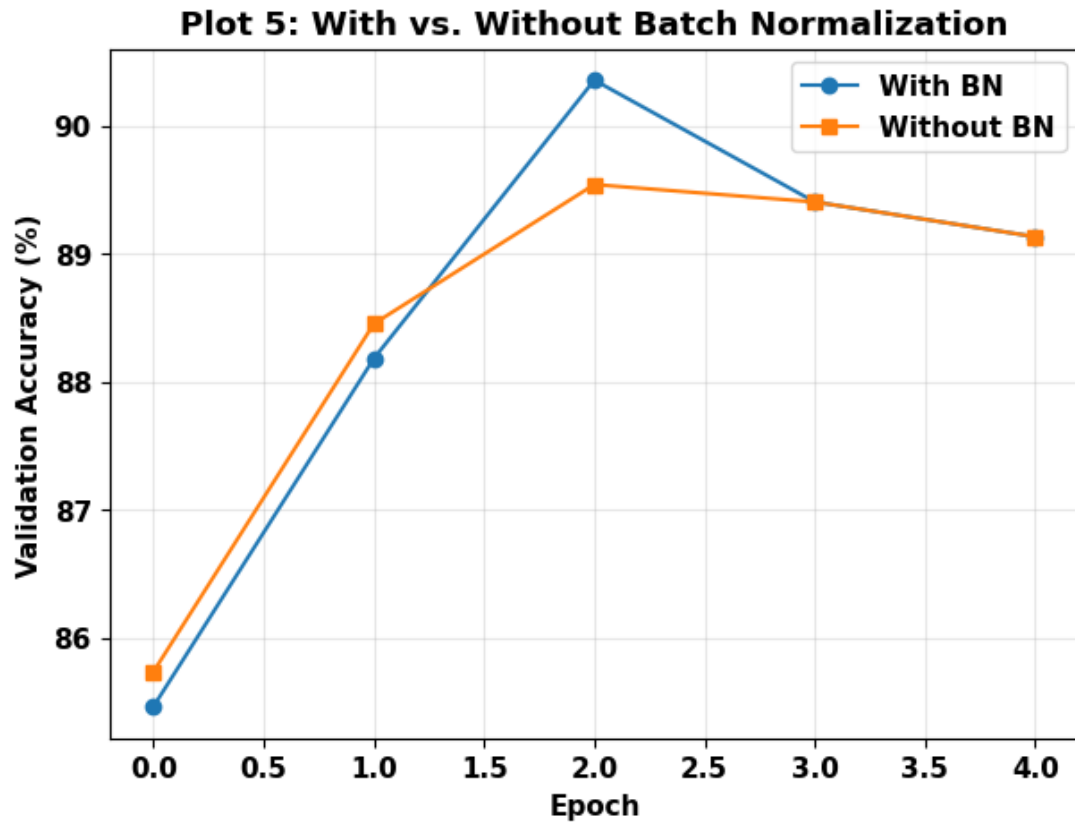


Figure 6: Validation accuracy with and without Batch Normalization.

Inference: Both configurations improve rapidly and finish at nearly the same validation accuracy. Batch Normalization reaches a higher peak near epoch 2, but the advantage disappears later. Normalizing intermediate activations can stabilize and accelerate early training, although the final result also depends on the data and other hyperparameters.

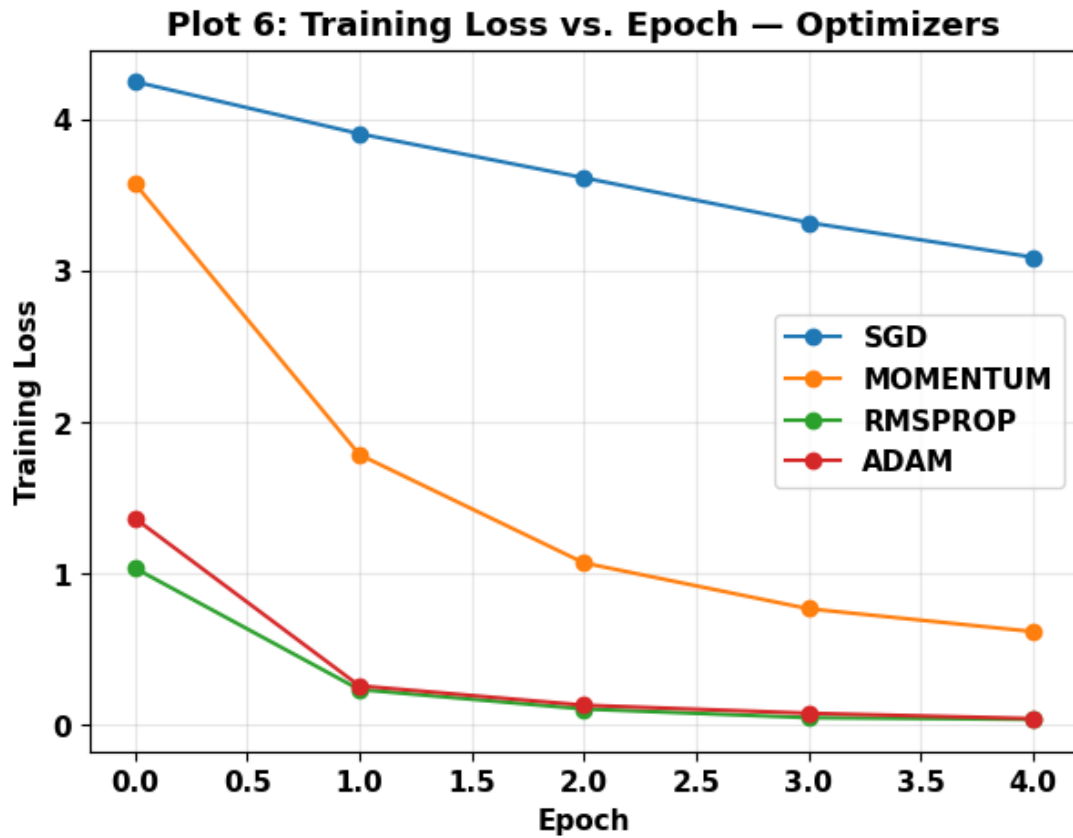


Figure 7: Training loss for the tested optimization algorithms.

Inference: RMSProp and Adam reduce training loss to near zero most quickly, Momentum converges more gradually, and plain SGD remains comparatively high. Adaptive learning rates help RMSProp and Adam respond efficiently to gradient scale, while Momentum accelerates consistent updates and SGD makes slower fixed-step progress.

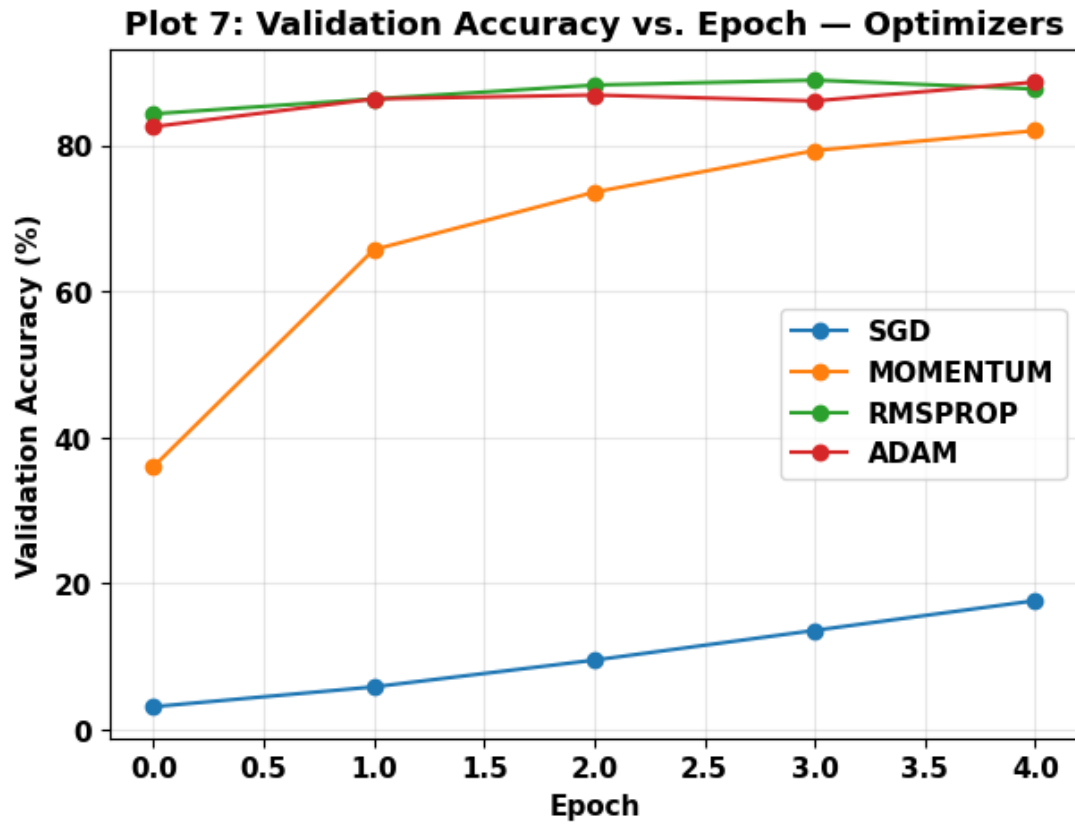


Figure 8: Validation accuracy for the tested optimization algorithms.

Inference: RMSProp and Adam maintain the highest validation accuracy, Momentum improves steadily, and SGD remains far below the other optimizers. Adaptive methods reach useful solutions quickly, Momentum accumulates a helpful update direction, and the selected SGD settings learn too slowly within the available epochs.

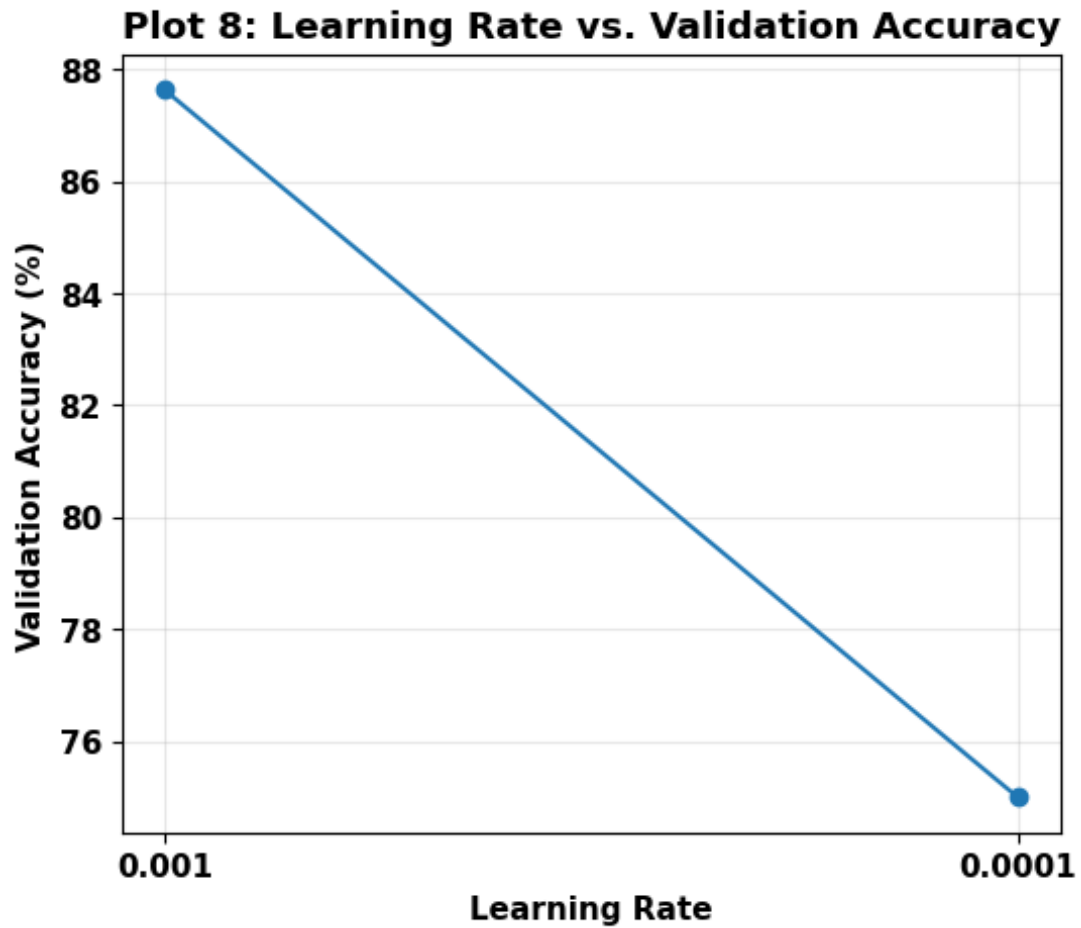


Figure 9: Validation accuracy for the tested learning rates.

Inference: The learning rate of 0.001 achieves approximately 87.6% validation accuracy, compared with about 75.0% for 0.0001. The smaller learning rate therefore converges too slowly within the fixed training duration. A larger but stable update size allows the model to reach a better solution in the available epochs.

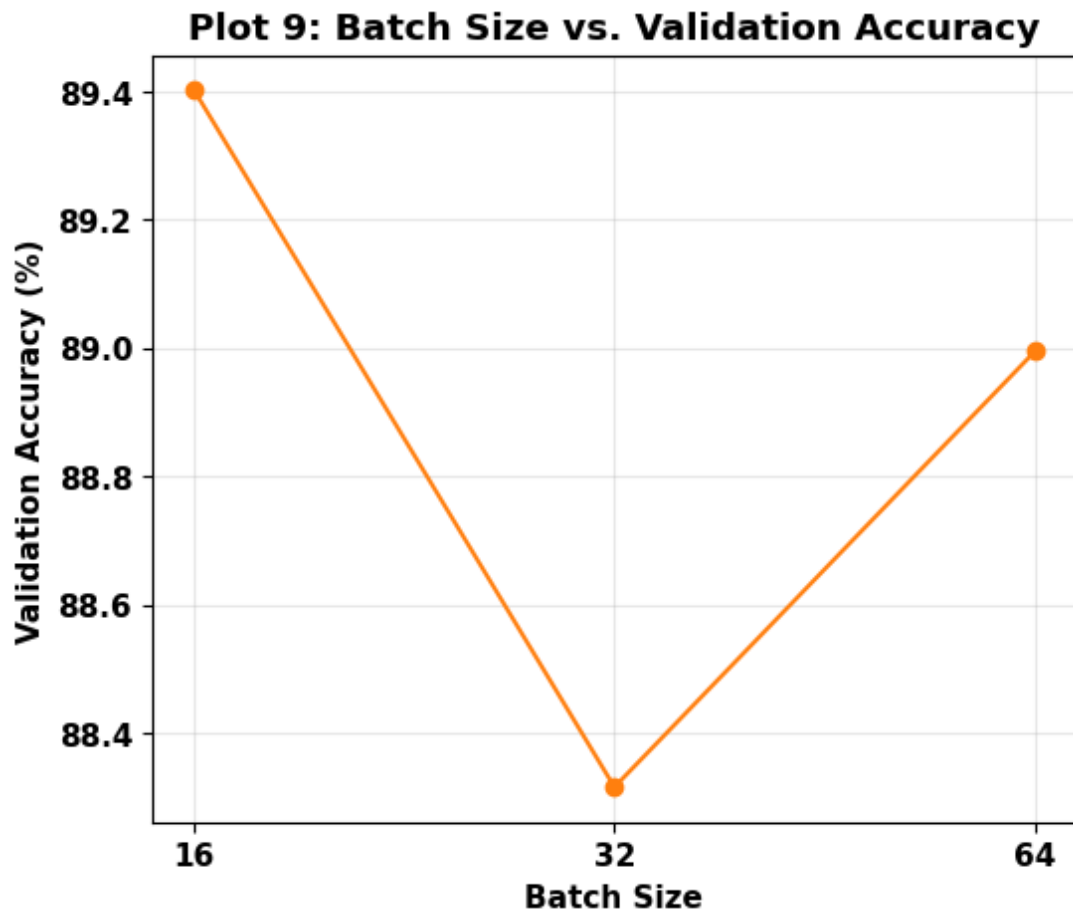


Figure 10: Validation accuracy for the tested batch sizes.

Inference: Batch size 16 produces the highest validation accuracy, batch size 32 gives the lowest, and batch size 64 lies between them. The overall differences are small, suggesting only a modest batch-size effect. Smaller batches can introduce helpful gradient noise, while observed fluctuations may also result from sampling and training variability.

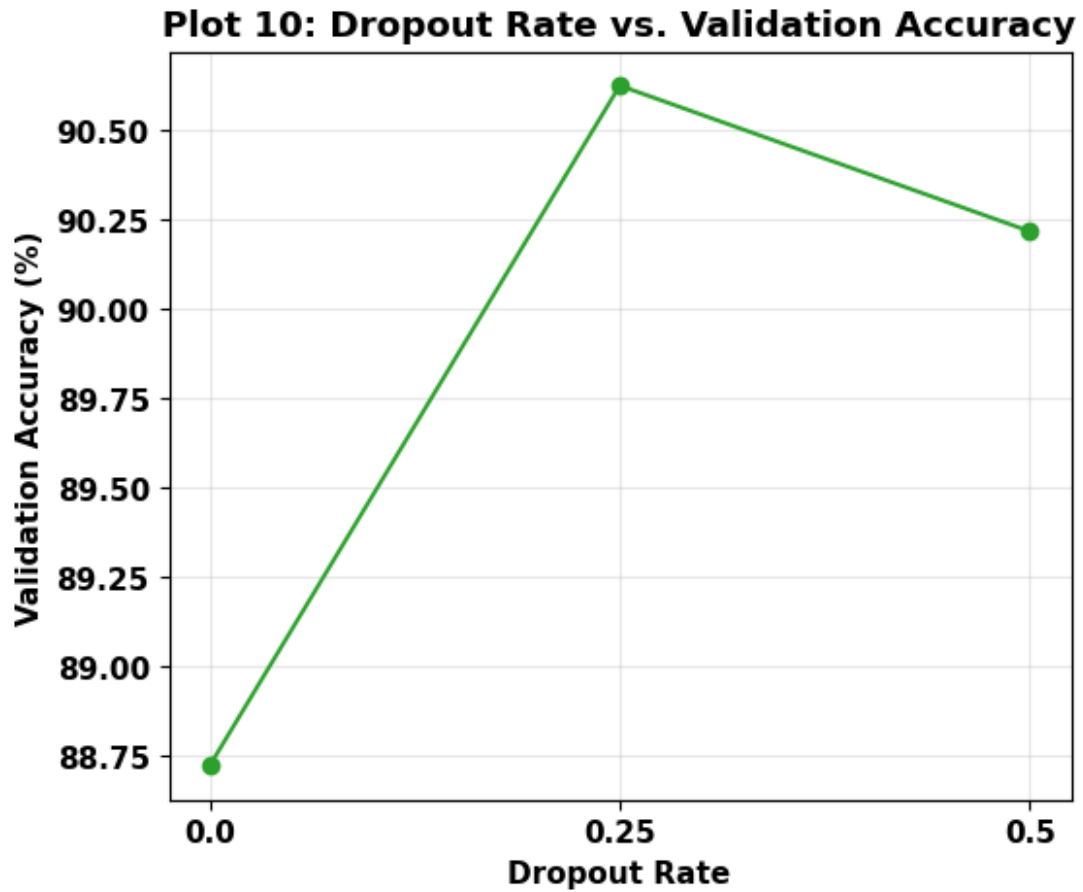


Figure 11: Validation accuracy for the tested Dropout rates.

Inference: A Dropout rate of 0.25 achieves the highest validation accuracy, followed by 0.5, while no Dropout performs worst. Moderate Dropout reduces co-adaptation and improves generalization. A rate of 0.5 removes more activations during training and may slightly restrict the model's learning capacity.

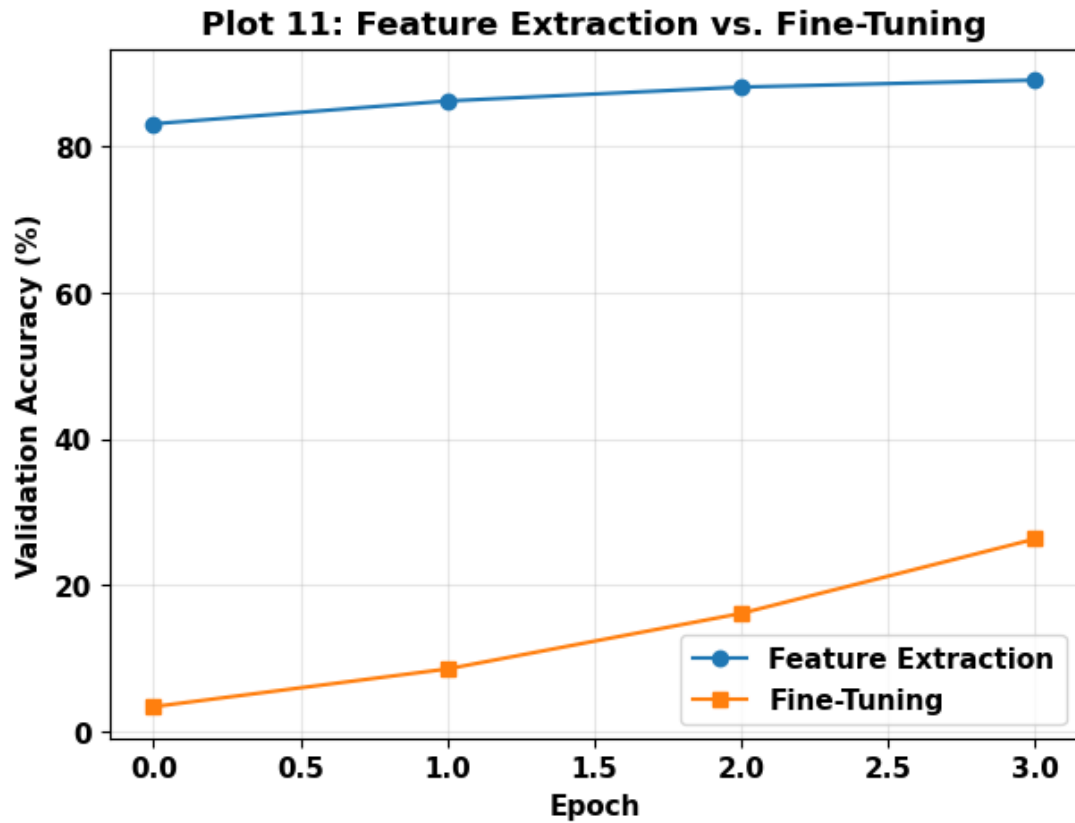


Figure 12: Validation accuracy during feature extraction and fine-tuning.

Inference: Feature extraction starts above 80% and improves slightly, whereas fine-tuning begins very low and rises gradually to about 26%. The frozen pretrained features already provide a strong representation, while the unfrozen layers require careful adaptation. More epochs or a better fine-tuning learning rate may be needed to preserve useful pretrained weights.

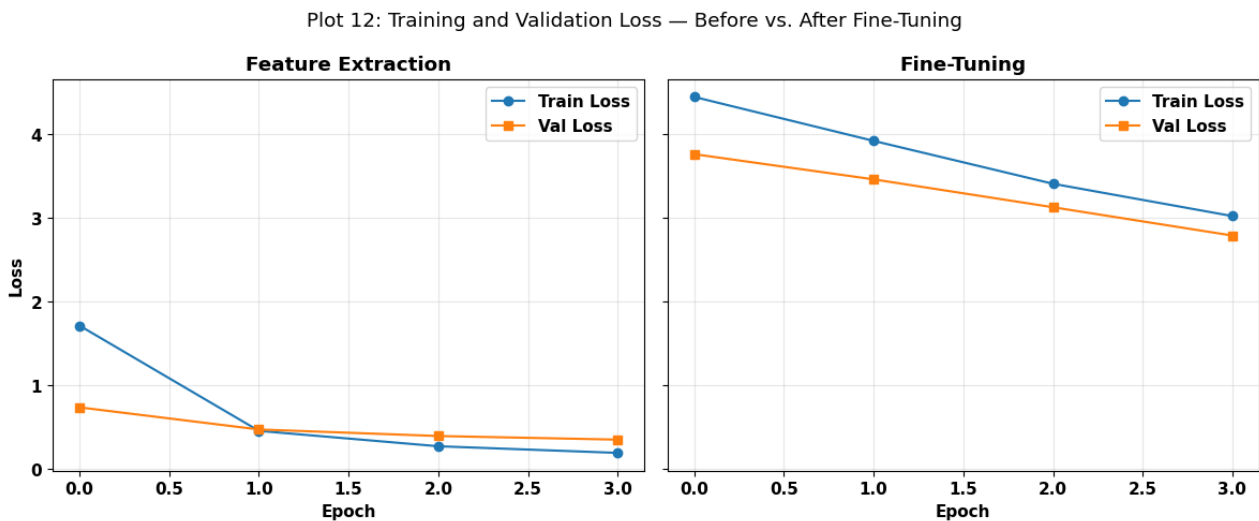


Figure 13: Training and validation loss before and after fine-tuning.

Inference: Feature extraction reduces both losses quickly to low values, while fine-tuning starts with much larger losses that decline steadily but remain high. The close training and validation curves suggest limited overfitting in both cases. Fine-tuning changes pretrained parameters and needs slower, longer optimization to adapt without erasing previously learned features.

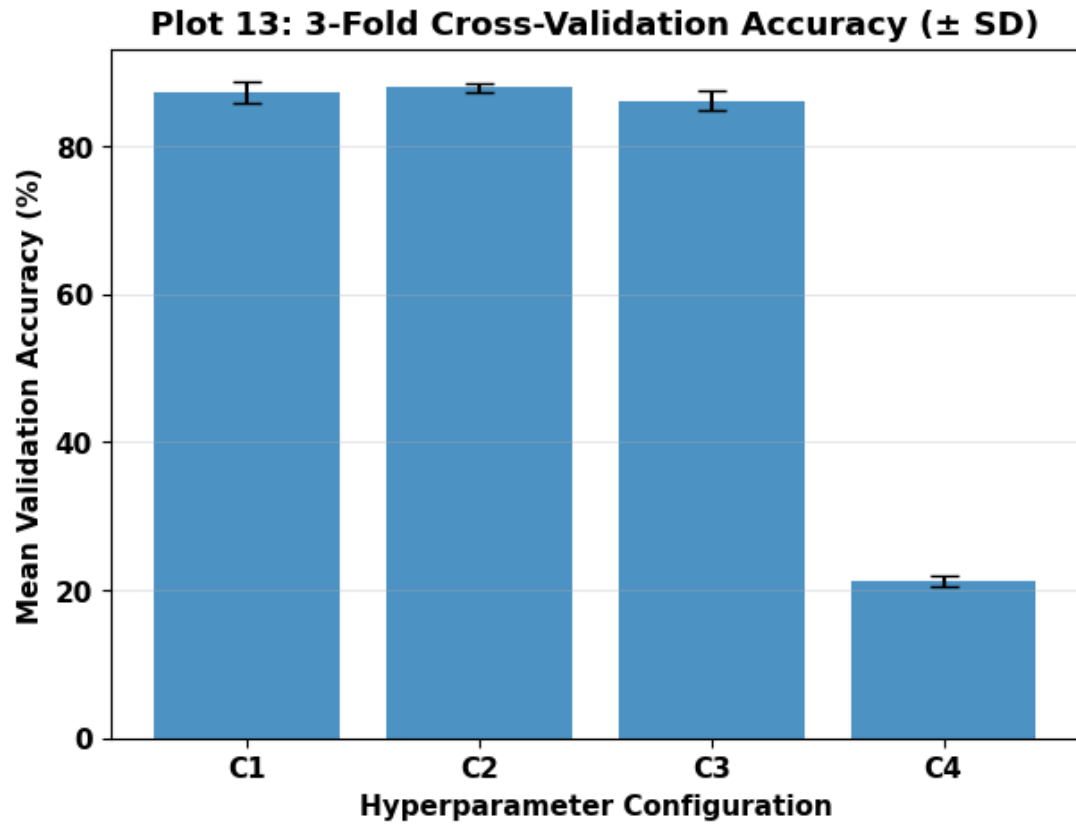


Figure 14: Three-fold cross-validation accuracy with standard-deviation error bars.

Inference: The plot compares the mean cross-validation accuracy and variability of configurations C1–C4. C2 achieves the highest mean accuracy and the smallest error bar, while C4 performs substantially worse than the other configurations. This suggests that C2 provides the strongest and most consistent generalization across the three folds.

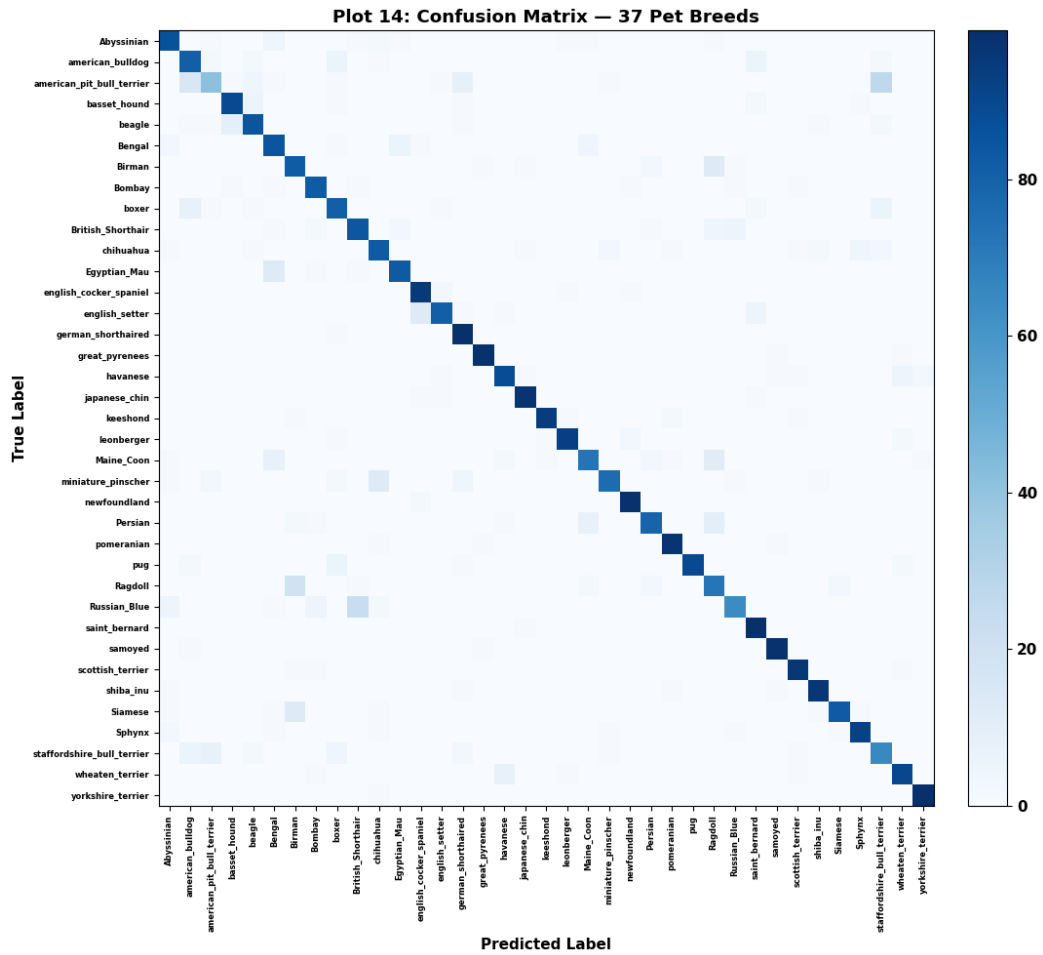


Figure 15: Confusion matrix for the 37 pet breeds.

Inference: The strong dark diagonal shows that most images are assigned to their correct breed, indicating good overall classification performance. The lighter off-diagonal cells reveal occasional confusion between visually similar cat or dog breeds. These errors can arise from shared coat patterns, body shapes, poses, or image backgrounds.

Optional Plot 15: Representative Misclassified Images

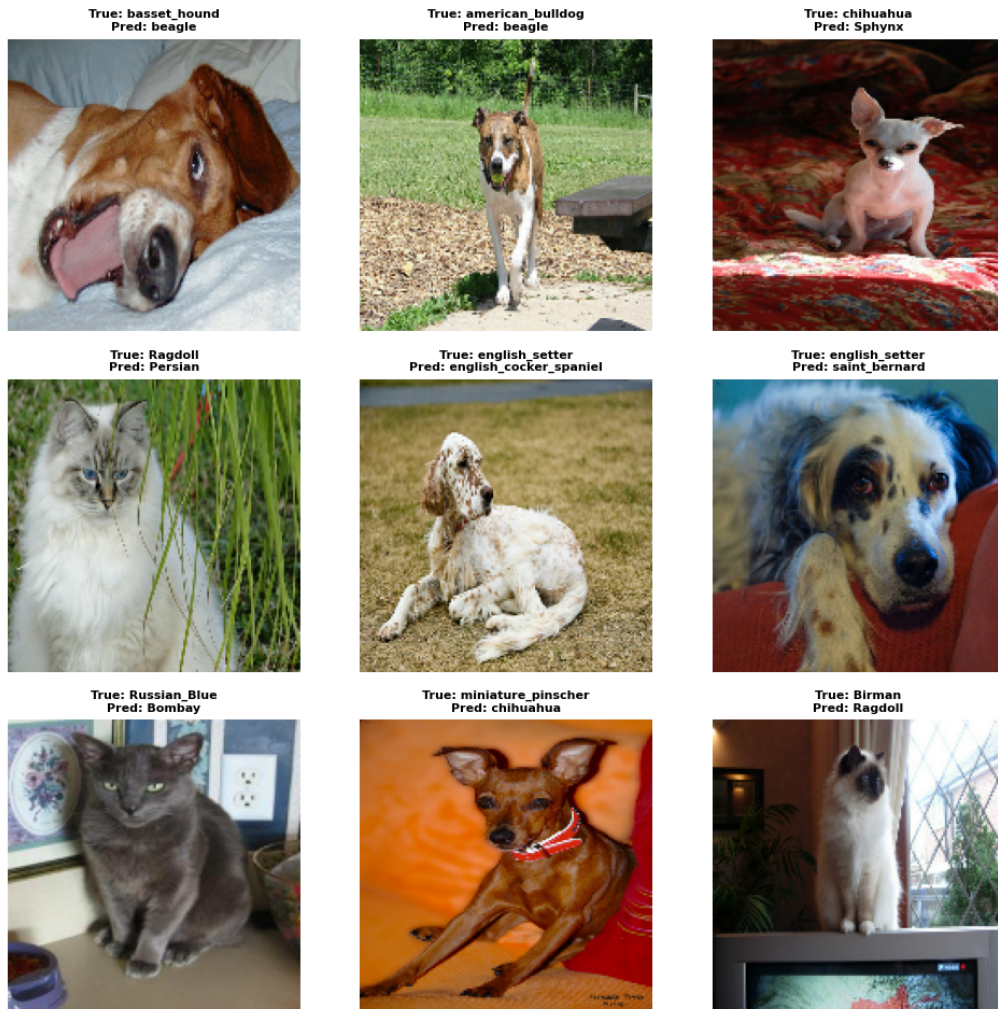


Figure 16: Representative images misclassified by the final model.

Inference: The plot shows examples in which the predicted breed differs from the true breed, including basset hound–beagle, Ragdoll–Persian, and Birman–Ragdoll confusions. Many pairs share similar facial structure, fur, coloring, or body proportions. Unusual poses, lighting, and background clutter can further hide breed-specific features and lead to incorrect predictions.

LAB 5- [github link](#)